ML Fundamentals

Feature Engineering, Statistics, and the Core Ideas

Bernard

May 25, 2026

"Machine learning is not magic. It is statistics plus computation plus a willingness to be wrong."

Contents

Part I

Foundations

Chapter 1

Introduction to Machine Learning

> "Machine learning is the science of getting computers to act without being explicitly programmed."
>
> Arthur Samuel, 1959

Learning Goals

- Define machine learning and distinguish it from traditional programming.
- Classify learning problems: supervised, unsupervised, reinforcement learning.
- Understand the bias-variance tradeoff as the central tension in ML.
- Formulate learning as function approximation from data.

1.1 What Is Machine Learning?

Traditional programming: a human writes explicit rules (a program) that maps inputs to outputs. Machine learning inverts this: the computer learns the rules from examples.

Definition 1.1 (Machine Learning)**.** A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

In practical terms: given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$, find a function f such that $f(\mathbf{x}) \approx y$ for new, unseen inputs.

1.2 The Three Learning Paradigms

Paradigm	Data	Goal
Supervised learning	Input–output pairs $(\mathbf{x}, y)$	Predict y from $\mathbf{x}$
Unsupervised learning	Inputs only $\{\mathbf{x}_i\}$	Discover hidden structure
Reinforcement learning	States, actions, rewards	Learn optimal policy

This book focuses on supervised and unsupervised learning, which form the foundation of most practical ML.

1.2.1 Supervised Learning

Given labeled examples $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, learn a mapping $f : \mathcal{X} \to \mathcal{Y}$:

- **Regression:** $\mathcal{Y} = \mathbb{R}$ (continuous output). Examples: house prices, temperature.
- **Classification:** $\mathcal{Y} = \{1, \ldots, K\}$ (discrete classes). Examples: spam detection, image recognition.

The quality of f is measured by a *loss function* $L(y, f(\mathbf{x}))$:

- Squared error for regression: $L(y, \hat{y}) = (y - \hat{y})^2$
- Cross-entropy for classification: $L(y, \hat{y}) = -\sum_k y_k \log \hat{y}_k$

1.2.2 Unsupervised Learning

Given unlabeled inputs $\{\mathbf{x}_i\}_{i=1}^n$, discover structure:

- **Clustering:** group similar points (K-means, DBSCAN).
- **Dimensionality reduction:** find a low-dimensional representation (PCA).
- **Density estimation:** model $p(\mathbf{x})$ directly.

1.3 The Central Tension: Bias–Variance Tradeoff

Every model makes a tradeoff between two sources of error:

Definition 1.2 (Bias and Variance)**.** The **bias** of an estimator measures how far the average prediction is from the true value. The **variance** measures how much the prediction changes across different training sets.

For a model f trained on dataset $\mathcal{D}$:

$$\mathbb{E}_{\mathcal{D}}[(y - f(\mathbf{x}; \mathcal{D}))^2] = \underbrace{\left(\mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] - \mathbb{E}[y|\mathbf{x}]\right)^2}_{\text{Bias}^2} + \underbrace{\text{Var}_{\mathcal{D}}\left(f(\mathbf{x}; \mathcal{D})\right)}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible error}}$$

- **High bias:** model is too simple, underfits the data (e.g., linear regression on a quadratic function).
- **High variance:** model is too complex, overfits the noise (e.g., high-degree polynomial).

The goal of feature engineering, regularization, and model selection is to find the sweet spot.

Principle 1.1 (No Free Lunch)**.** No single algorithm works best for all problems. Every model encodes assumptions (inductive bias). The art of ML is matching the bias to the data.

1.4 Learning as Function Approximation

From a statistical perspective, all of supervised learning reduces to:

$$y = f^*(\mathbf{x}) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

where f^* is the true (unknown) function and ϵ is irreducible noise. We choose a hypothesis class $\mathcal{H}$ (e.g., linear functions, decision trees) and search for the best $f \in \mathcal{H}$ that minimizes empirical risk:

$$\hat{f} = \arg\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i))$$

The gap between $\hat{f}$ and f^* depends on:

1. **Approximation error:** how well $\mathcal{H}$ can represent f^*.
2. **Estimation error:** how well $\hat{f}$ approximates the best in $\mathcal{H}$ given finite data.
3. **Optimization error:** how well our algorithm finds $\hat{f}$.

1.5 The Role of Feature Engineering

No amount of statistical sophistication compensates for poor features. Feature engineering—transforming raw data into informative predictors—is often the highest-leverage activity in applied ML.

> "Coming up with features is difficult, time-consuming, requires expert knowledge. Applied machine learning is basically feature engineering." — Andrew Ng

This book devotes Chapter 6 entirely to this topic.

1.6 Exercises

Exercise 1.1. *For each of the following tasks, state whether it is supervised or unsupervised learning, and whether it is regression or classification:*

- *Predicting tomorrow's stock price.*
- *Grouping customer purchase histories into segments.*
- *Detecting fraudulent transactions from labeled examples.*
- *Compressing a dataset to 50 dimensions.*

Exercise 1.2. *A linear model has high bias and low variance. A 15th-degree polynomial has low bias and high variance. Sketch the bias-variance decomposition as model complexity increases. Where is the optimal point?*

Exercise 1.3. *Suppose the true function is $f^*(x) = \sin(x)$ and we fit a linear model $f(x) = ax + b$. Decompose the expected test error at $x = \pi/2$ into bias and variance components (conceptually; numerical computation not required).*

Exercise 1.4. *Explain why the No Free Lunch theorem implies that we must use domain knowledge to choose a model class. Provide a concrete example.*

Chapter 2

Probability and Statistics for Machine Learning

> "All models are wrong, but some are useful."
>
> George E. P. Box [1]

Learning Goals

- Review the core probability concepts: random variables, distributions, expectation, Bayes' rule.
- Understand maximum likelihood and maximum a posteriori estimation.
- Distinguish between parametric and non-parametric methods.
- Apply hypothesis testing and confidence intervals to model comparison.

2.1 Random Variables and Distributions

A *random variable* X is a variable whose value is determined by a random process. The *probability distribution* describes how likely each value is.

Definition 2.1 (Probability Distribution)**.** A **probability distribution** assigns a probability to each possible outcome of a random variable. For discrete X, we use a probability mass function $p(x) = P(X = x)$. For continuous X, we use a probability density function $f(x)$ where $P(a \leq X \leq b) = \int_a^b f(x)\,dx$.

2.1.1 Key Distributions for ML

Distribution	Parameters	Uses in ML
Bernoulli(p)	$p \in [0, 1]$	Binary classification (coin flip)
Binomial(n, p)	n trials, p success	Count of successes
Normal$\mathcal{N}(\mu, \sigma^2)$	mean μ, variance σ^2	Noise model, continuous targets
Exponential(λ)	rate λ	Waiting times
Poisson(λ)	rate λ	Count data
Beta(α, β)	shape α, β	Prior for Bernoulli/Binomial

$\text{Gamma}(k, \theta)$	shape k, scale θ	Prior for precision

The normal distribution deserves special attention. Its density is:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

2.1.2 Expectation and Variance

Definition 2.2 (Expectation)**.** The **expected value** of a function g of random variable X is:

$$\mathbb{E}[g(X)] = \begin{cases} \sum_x g(x)p(x) & \text{if } X \text{ is discrete} \\ \int g(x)f(x)\,dx & \text{if } X \text{ is continuous} \end{cases}$$

Key properties:

- $\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$
- $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$
- $\text{Var}(aX + b) = a^2 \text{Var}(X)$
- $\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]$

2.2 Multivariate Statistics

For a d-dimensional random vector $\mathbf{X} = (X_1, \ldots, X_d)$:

- **Mean vector:** $\boldsymbol{\mu} = \mathbb{E}[\mathbf{X}]$
- **Covariance matrix:** $\Sigma_{ij} = \text{Cov}(X_i, X_j)$

The multivariate normal distribution:

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

This is the building block for Gaussian mixture models, linear discriminant analysis, and Gaussian processes.

2.3 Bayes' Rule and Probabilistic Inference

Bayes' rule is the foundation of probabilistic ML:

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)\, p(\theta)}{p(\mathcal{D})}$$

- $p(\theta)$: **prior** — what we believe about θ before seeing data.
- $p(\mathcal{D} \mid \theta)$: **likelihood** — how probable the data is given θ.
- $p(\theta \mid \mathcal{D})$: **posterior** — updated belief after seeing data.
- $p(\mathcal{D})$: **evidence** — marginal probability of the data.

Example 2.1 (Coin Toss)**.** Suppose we flip a coin 10 times and observe 7 heads. We model the coin's bias $\theta = P(\text{heads})$ with a $\text{Beta}(2, 2)$ prior (peaked at 0.5). The posterior is:

$$p(\theta \mid 7 \text{ heads}) \propto \text{Binomial}(7 \mid 10, \theta) \times \text{Beta}(\theta \mid 2, 2)$$

$$= \text{Beta}(\theta \mid 2 + 7, 2 + 3) = \text{Beta}(\theta \mid 9, 5)$$

The posterior mean is $\frac{9}{14} \approx 0.64$, shifted from the prior mean of 0.5 toward the observed 0.7.

2.4 Maximum Likelihood Estimation

The most common estimation principle: choose θ to maximize the likelihood of the data.

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} p(\mathcal{D} \mid \theta) = \arg\max_{\theta} \sum_{i=1}^{n} \log p(y_i \mid \mathbf{x}_i, \theta)$$

Example 2.2 (MLE for Linear Regression)**.** Assume $y_i = \mathbf{w}^\top \mathbf{x}_i + \epsilon_i$, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. The log-likelihood is:

$$\log p(\mathcal{D} \mid \mathbf{w}) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{n} (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

Maximizing this is equivalent to minimizing $\sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$ — ordinary least squares.

2.5 Maximum a Posteriori Estimation

MAP estimation adds a prior:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \log p(\theta \mid \mathcal{D}) = \arg\max_{\theta} \left[\log p(\mathcal{D} \mid \theta) + \log p(\theta) \right]$$

A Gaussian prior on $\mathbf{w}$ (with zero mean) is equivalent to L2 regularization:

$$\log p(\mathbf{w}) = -\frac{\lambda}{2} \|\mathbf{w}\|_2^2 + \text{const}$$

Thus, ridge regression is the MAP estimate under a Gaussian prior.

2.6 Hypothesis Testing and Confidence Intervals

2.6.1 Null Hypothesis Significance Testing

When comparing two models A and B:

1. **Null hypothesis** H_0: models A and B have equal performance.
2. Compute a test statistic (e.g., difference in accuracy).
3. Compute the p-value: probability of observing the statistic (or more extreme) under H_0.
4. If $p < \alpha$ (typically 0.05), reject H_0.

Definition 2.3 (p-value)**.** The **p-value** is the probability of obtaining results at least as extreme as those observed, assuming the null hypothesis is true. It is NOT the probability that H_0 is true.

2.6.2 Confidence Intervals

A 95% confidence interval for a parameter θ is an interval $[L, U]$ such that, if we repeated the experiment many times, 95% of the intervals would contain the true θ.

For model accuracy $\hat{a}$, an approximate confidence interval is:

$$\hat{a} \pm z_{\alpha/2} \sqrt{\frac{\hat{a}(1-\hat{a})}{n}}$$

where $z_{\alpha/2} = 1.96$ for 95% confidence.

2.7 Information-Theoretic Concepts

2.7.1 Entropy

The entropy of a discrete random variable X measures its uncertainty:

$$H(X) = -\sum_{x} p(x) \log p(x)$$

- $H(X) = 0$ if X is deterministic.
- $H(X)$ is maximized for a uniform distribution.

2.7.2 Kullback-Leibler Divergence

The KL divergence measures how one distribution p differs from another q:

$$D_{\mathrm{KL}}(p \parallel q) = \sum_{x} p(x) \log \frac{p(x)}{q(x)}$$

- $D_{\mathrm{KL}}(p \parallel q) \geq 0$, with equality iff $p = q$.
- Not symmetric: $D_{\mathrm{KL}}(p \parallel q) \neq D_{\mathrm{KL}}(q \parallel p)$.
- Used as the loss function in logistic regression and neural network classification (cross-entropy).

2.8 Exercises

Exercise 2.1. *Prove that* $\mathrm{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$.

Exercise 2.2. *You collect 100 data points from* $\mathcal{N}(\mu, 1)$ *and compute* $\bar{x} = 3.2$. *Compute a* 95% *confidence interval for* μ *and interpret it. What would happen to the interval width if you collected 400 points?*

Exercise 2.3. *Derive the MLE for the parameter* λ *of a Poisson distribution:* $p(k \mid \lambda) = \frac{\lambda^k e^{-\lambda}}{k!}$. *Given data* $\{k_1, \ldots, k_n\}$, *what is* $\hat{\lambda}_{MLE}$*?*

Exercise 2.4. *Two classifiers achieve accuracies of* 90% *and* 92% *on a test set of 500 samples. Is the difference statistically significant at* $\alpha = 0.05$*? Compute the p-value using a normal approximation.*

Chapter 3

Linear Algebra and Optimization

"Statistics is the science of data. Linear algebra is the language of statistics."

Learning Goals

- Review the essential linear algebra: vectors, matrices, eigenvalues, SVD.
- Understand gradients, convexity, and gradient-based optimization.
- Derive the normal equations for linear regression.
- Connect matrix factorizations to dimensionality reduction.

3.1 Vectors and Matrices

A vector $\mathbf{x} \in \mathbb{R}^d$ is an ordered list of d numbers. A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a rectangular array.

3.1.1 Key Operations

- **Inner product:** $\mathbf{x}^\top \mathbf{y} = \sum_{i=1}^{d} x_i y_i$
- **Norm:** $\|\mathbf{x}\|_p = (\sum_i |x_i|^p)^{1/p}$, with $\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}}$
- **Matrix–vector product:** $(\mathbf{A}\mathbf{x})_i = \sum_j A_{ij} x_j$
- **Matrix–matrix product:** $(\mathbf{A}\mathbf{B})_{ij} = \sum_k A_{ik} B_{kj}$
- **Transpose:** $(\mathbf{A}^\top)_{ij} = A_{ji}$
- **Trace:** $\operatorname{tr}(\mathbf{A}) = \sum_i A_{ii}$

3.1.2 Special Matrices

Type	Definition	Property
Symmetric	$\mathbf{A} = \mathbf{A}^\top$	Real eigenvalues
Positive definite	$\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$ for $\mathbf{x} \neq 0$	All eigenvalues positive
Orthogonal	$\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$	Preserves norms
Diagonal	$A_{ij} = 0$ for $i \neq j$	Easy to invert

3.2 Eigenvalues and Eigenvectors

Definition 3.1 (Eigenvalue Decomposition)**.** For a square matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$, $\mathbf{v}$ is an **eigenvector** with **eigenvalue** λ if:

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$$

Equivalently: $\mathbf{AV} = \mathbf{V}\mathbf{\Lambda}$, where $\mathbf{V}$'s columns are eigenvectors and $\mathbf{\Lambda}$ is diagonal with eigenvalues.

Key facts:

- $\operatorname{tr}(\mathbf{A}) = \sum_i \lambda_i$
- $\det(\mathbf{A}) = \prod_i \lambda_i$
- A symmetric matrix can be diagonalized as $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ with orthogonal $\mathbf{Q}$.
- The eigenvalues of $\mathbf{A}^{-1}$ are $1/\lambda_i$.

Eigenvalues are central to PCA (Chapter 7), where the eigenvectors of the covariance matrix give the principal components.

3.3 Singular Value Decomposition

The SVD generalizes eigenvalue decomposition to non-square matrices.

Definition 3.2 (SVD)**.** Any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ can be factored as:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

where $\mathbf{U} \in \mathbb{R}^{m \times m}$ and $\mathbf{V} \in \mathbb{R}^{n \times n}$ are orthogonal, and $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is diagonal with $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(m,n)} \geq 0$.

The singular values σ_i and the SVD are the mathematical foundation for:

- **PCA:** the right singular vectors $\mathbf{V}$ are the principal components.
- **Matrix approximation:** keeping only the top k singular values gives the best rank-k approximation (Eckart–Young theorem).
- **Pseudoinverse:** $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\top$ solves least squares.
- **Rank and condition number:** $\operatorname{rank}(\mathbf{A}) = \#$ nonzero σ_i, $\kappa(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$.

3.4 Calculus and Gradients

3.4.1 Derivatives of Vector Functions

The gradient of a scalar function $f : \mathbb{R}^d \to \mathbb{R}$ is:

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1} & \cdots & \frac{\partial f}{\partial x_d} \end{pmatrix}^\top$$

The Hessian $\nabla^2 f(\mathbf{x})$ is the $d \times d$ matrix of second derivatives.

3.4.2 Useful Matrix Derivatives

- $\nabla_{\mathbf{x}}(\mathbf{a}^\top \mathbf{x}) = \mathbf{a}$
- $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x}$
- $\nabla_{\mathbf{x}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 = 2\mathbf{A}^\top(\mathbf{A}\mathbf{x} - \mathbf{b})$
- $\nabla_{\mathbf{X}} \operatorname{tr}(\mathbf{A}\mathbf{X}) = \mathbf{A}^\top$

Example 3.1 (Deriving the Normal Equations)**.** Minimize $L(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$:

$$\nabla L = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) = 0$$
$$\mathbf{X}^\top\mathbf{X}\mathbf{w} = \mathbf{X}^\top\mathbf{y}$$
$$\hat{\mathbf{w}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$$

3.5 Convexity

Definition 3.3 (Convex Function)**.** A function $f : \mathbb{R}^d \to \mathbb{R}$ is **convex** if for all $\mathbf{x}, \mathbf{y}$ and $\lambda \in [0, 1]$:

$$f(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1-\lambda)f(\mathbf{y})$$

Equivalently (if twice differentiable): $\nabla^2 f(\mathbf{x}) \succeq 0$ (positive semidefinite Hessian).

Convexity guarantees that any local minimum is a global minimum. Linear regression, logistic regression, and SVM (with convex loss) all optimize convex functions. Neural networks are non-convex.

3.6 Gradient Descent

When the normal equations are not available (e.g., logistic regression, neural networks), we use iterative optimization.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\nabla L(\mathbf{w}^{(t)})$$

3.6.1 Learning Rate

The learning rate η controls step size:

- Too large: divergence.
- Too small: slow convergence.
- Adaptive methods (Adam, RMSprop) adjust η per parameter.

3.6.2 Stochastic Gradient Descent

Instead of computing the gradient on all n data points (which is $O(nd)$ per step), SGD approximates it with a single data point:

$$\nabla L(\mathbf{w}) \approx \nabla\ell(y_i, f(\mathbf{x}_i; \mathbf{w}))$$

Mini-batch SGD uses B points (e.g., $B = 32$), balancing noise and computation.

Principle 3.1 (SGD Convergence)**.** SGD converges to a global minimum for convex losses and to a good local minimum for non-convex losses, provided the learning rate decays appropriately: $\sum_t \eta_t = \infty$ and $\sum_t \eta_t^2 < \infty$.

3.7 The Chain Rule and Backpropagation

For nested functions $L = \ell(f(g(\mathbf{x}; \mathbf{w})))$:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial f_k} \cdot \frac{\partial f_k}{\partial g_j} \cdot \frac{\partial g_j}{\partial w_{ij}}$$

This is the chain rule, applied systematically through a computation graph. Backpropagation is simply the chain rule with caching of intermediate results.

3.8 Exercises

Exercise 3.1. *Compute the eigenvalues and eigenvectors of* $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$.

Exercise 3.2. *Prove that* $\|\mathbf{A}\mathbf{x}\|_2 \leq \sigma_{\max}(\mathbf{A})\|\mathbf{x}\|_2$ *for any vector* $\mathbf{x}$.

Exercise 3.3. *Take one step of gradient descent on* $f(x) = x^4 - 3x^3 + 2$ *starting at* $x = 2$ *with* $\eta = 0.01$. *What is the new value of* x*?*

Exercise 3.4. *Show that the sum of convex functions is convex. Use this to explain why the empirical risk for logistic regression is convex.*

Part II

Core Methods

Chapter 4

Regression

"All of machine learning is either regression or classification. Everything else is a trick."

Learning Goals

- Derive and interpret linear regression via ordinary least squares.
- Understand the assumptions underlying linear regression and how violations affect inference.
- Extend linear regression with basis functions for non-linear relationships.
- Diagnose model fit using residual analysis and influence measures.

4.1 Linear Regression

The workhorse of supervised learning: model the target as a linear combination of features.

$$y = \mathbf{w}^\top \mathbf{x} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

Given data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, collect inputs into matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ and outputs into vector $\mathbf{y} \in \mathbb{R}^n$:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

4.1.1 Ordinary Least Squares

The OLS estimator minimizes the sum of squared residuals:

$$\hat{\mathbf{w}}_{\text{OLS}} = \arg\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

Setting the gradient to zero gives the *normal equations*:

$$\mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

$$\hat{\mathbf{w}}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Definition 4.1 (Fitted Values and Residuals)**. Fitted values:** $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}} = \mathbf{H}\mathbf{y}$, where $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$ is the **hat matrix**.
Residuals: $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$.

4.1.2 Geometric Interpretation

OLS projects $\mathbf{y}$ onto the column space of $\mathbf{X}$. The hat matrix $\mathbf{H}$ is the projection matrix. The residuals $\mathbf{e}$ are orthogonal to $\text{col}(\mathbf{X})$:

$$\mathbf{X}^\top\mathbf{e} = \mathbf{0}$$

4.2 Assumptions and Diagnostics

Linear regression makes four key assumptions:

Assumption	Meaning	Diagnostic
Linearity	$\mathbb{E}[y \mid \mathbf{x}]$ is linear in $\mathbf{x}$	Residuals vs. fitted plot
Independence	Observations are independent	Durbin–Watson test
Homoscedasticity	$\text{Var}(\epsilon \mid \mathbf{x}) = \sigma^2$ (constant)	Scale–location plot
Normality	$\epsilon \sim \mathcal{N}(0, \sigma^2)$	Q–Q plot of residuals

4.2.1 Influential Points

A point is *influential* if removing it substantially changes $\hat{\mathbf{w}}$. Measure with Cook's distance:

$$D_i = \frac{e_i^2}{d \cdot \text{MSE}} \cdot \frac{h_{ii}}{(1 - h_{ii})^2}$$

where h_{ii} is the i-th diagonal of $\mathbf{H}$ (the *leverage*). A common threshold: $D_i > 4/n$ warrants investigation.

4.3 Basis Function Expansion

Linear regression is limited to linear relationships—but only the *parameters* must be linear. The features can be non-linear transformations:

$$y = w_0 + w_1\phi_1(x) + w_2\phi_2(x) + \cdots + w_k\phi_k(x) + \epsilon$$

4.3.1 Polynomial Basis

$$\phi_j(x) = x^j$$

Listing 4.1: Polynomial regression

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X)
model = LinearRegression().fit(X_poly, y)
```

Choosing the polynomial degree is a bias–variance tradeoff. Low degrees underfit; high degrees overfit.

4.3.2 Spline Basis

Splines are piecewise polynomials joined at *knots*. Cubic splines are the most common: each piece is a cubic polynomial, with continuity constraints at knots.

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \theta_k (x - \xi_k)_+^3 + \epsilon$$

where $(z)_+ = \max(0, z)$ and ξ_k are the knot locations. Splines are more stable than high-degree polynomials.

4.3.3 Regression Splines vs. Smoothing Splines

- **Regression splines:** choose knot locations and basis functions, then fit by OLS.
- **Smoothing splines:** place a knot at every data point and add a penalty on the second derivative to prevent overfitting.

4.4 Model Selection in Regression

4.4.1 Adjusted R^2

$$R^2_{\text{adj}} = 1 - \frac{\text{RSS}/(n - d - 1)}{\text{TSS}/(n - 1)}$$

Penalizes adding useless features, unlike plain R^2.

4.4.2 Information Criteria

- **AIC:** $-2 \log L + 2d$
- **BIC:** $-2 \log L + d \log n$

Both balance fit (log-likelihood) against model complexity. BIC penalizes complexity more heavily (useful for selecting the true model when it exists). AIC is better for prediction.

4.5 Multiple Regression and Interaction Effects

When features interact, their joint effect is not the sum of individual effects:

$$y = w_0 + w_1 x_1 + w_2 x_2 + w_{12} x_1 x_2 + \epsilon$$

The interaction term $x_1 x_2$ allows the effect of x_1 to depend on x_2:

$$\frac{\partial y}{\partial x_1} = w_1 + w_{12} x_2$$

Including all pairwise interactions adds $d(d-1)/2$ terms. Feature engineering often involves discovering important interactions.

4.6 Exercises

Exercise 4.1. *Derive the normal equations by setting the gradient of $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ to zero. Show each step.*

Exercise 4.2. *A dataset has 3 features and 100 observations. You fit a linear regression and obtain $R^2 = 0.85$ and $R^2_{adj} = 0.84$. You add 10 random noise features and refit. What do you expect to happen to R^2 and R^2_{adj}?*

Exercise 4.3. *Generate data from $y = \sin(x) + \epsilon$ with $x \in [0, 2\pi]$. Fit polynomials of degree 1, 3, 5, 15. Plot the fitted curves and report the test MSE on a held-out set. At what degree does overfitting begin?*

Exercise 4.4. *Explain why using a high-degree polynomial can produce wild extrapolations beyond the data range, while cubic splines are better behaved. What property of polynomials causes this?*

Chapter 5

Classification

"Classification is not regression with a different loss. Classification is a fundamentally different problem with different evaluation metrics and failure modes."

Learning Goals

- Distinguish between logistic regression, LDA, decision trees, and KNN for classification.
- Understand the probabilistic interpretation of logistic regression.
- Evaluate classifiers using appropriate metrics: accuracy, precision–recall, ROC–AUC.
- Handle class imbalance through resampling and cost-sensitive learning.

5.1 Logistic Regression

Despite its name, logistic regression is a classification method. It models the probability of class membership:

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

Definition 5.1 (Logistic (Sigmoid) Function)**.**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Maps $\mathbb{R} \to (0, 1)$ and is monotonic, with $\sigma(0) = 0.5$.

The decision boundary $\mathbf{w}^\top \mathbf{x} = 0$ is linear in feature space. Points with $\mathbf{w}^\top \mathbf{x} > 0$ are classified as class 1; otherwise class 0.

5.1.1 Loss Function

Logistic regression minimizes the *binary cross-entropy* (also called log loss):

$$L(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

This is the negative log-likelihood under a Bernoulli model. Unlike squared error for classification, cross-entropy heavily penalizes confident wrong predictions.

5.1.2 Gradient and Optimization

The gradient of the cross-entropy loss:

$$\nabla L(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i)\mathbf{x}_i$$

This has the same form as the OLS gradient but with $\hat{y}_i$ transformed through the sigmoid. There is no closed-form solution; we use gradient descent or Newton's method.

5.2 Linear Discriminant Analysis

LDA takes a different approach: model the class-conditional distributions $p(\mathbf{x} \mid y = k)$ as Gaussians with a shared covariance matrix.

$$p(\mathbf{x} \mid y = k) = \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma})$$

Using Bayes' rule:

$$p(y = k \mid \mathbf{x}) \propto p(\mathbf{x} \mid y = k)\, p(y = k)$$

The decision boundary between classes k and ℓ is linear in $\mathbf{x}$:

$$\boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} - \frac{1}{2}\boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k + \log \pi_k = \boldsymbol{\mu}_\ell^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} - \frac{1}{2}\boldsymbol{\mu}_\ell^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_\ell + \log \pi_\ell$$

where $\pi_k = p(y = k)$.

When the covariance matrices are allowed to differ per class (each class has its own $\boldsymbol{\Sigma}_k$), the decision boundaries become quadratic (QDA).

5.3 Decision Trees

Decision trees partition the feature space with axis-aligned splits.

Listing 5.1: Training a decision tree

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=3,
                               min_samples_split=10)
model.fit(X, y)
```

5.3.1 Split Criteria

At each node, the algorithm chooses the feature and threshold that maximize the information gain:

- **Gini impurity:** $G = \sum_{k=1}^{K} p_k(1 - p_k)$
- **Entropy:** $H = -\sum_{k=1}^{K} p_k \log p_k$
- **Misclassification rate:** $E = 1 - \max_k p_k$

The Gini impurity and entropy are preferred because they are differentiable and more sensitive to changes in node purity.

5.3.2 Strengths and Weaknesses

Trees are:

- Interpretable (if shallow).
- Non-parametric (no distributional assumptions).
- Handle non-linear relationships and interactions automatically.

But they:

- Have high variance (a small change in data can change the tree).
- Are prone to overfitting without pruning.
- Create piecewise constant predictions (not smooth).

5.4 K-Nearest Neighbors

KNN is a non-parametric method that memorizes the training data:

$$\hat{y}(\mathbf{x}) = \text{majority vote of the } k \text{ nearest neighbors}$$

Definition 5.2 (KNN)**. K-nearest neighbors** classifies a point based on the most common class among its k closest training points, where distance is typically Euclidean.

KNN has no training step (it is a *lazy learner*). The key hyperparameter is k:

- $k = 1$: zero training error, highly overfitted.
- $k = n$: always predicts the majority class.
- Optimal k balances bias and variance, typically $\sqrt{n}$ as a rule of thumb.

5.5 Multiclass Classification

Most binary classifiers extend to $K > 2$ classes:

- **One-vs-Rest (OvR):** train K binary classifiers, each distinguishing one class from the rest. Predict the class with the highest confidence.
- **Softmax regression (multinomial logistic regression):** directly models $p(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^\top \mathbf{x}}}$.

5.6 Evaluation Metrics

Accuracy is insufficient for most real-world problems.

Metric	Definition	When to use
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Balanced classes
Precision	$\frac{TP}{TP+FP}$	Minimize false positives
Recall	$\frac{TP}{TP+FN}$	Minimize false negatives
F_1	$2 \cdot \frac{P \cdot R}{P+R}$	Balanced precision–recall
ROC–AUC	Area under ROC curve	General discrimination
Log loss	$-\frac{1}{n}\sum[y \log \hat{y} + (1-y)\log(1-\hat{y})]$	Probabilistic calibration

5.7 Class Imbalance

When one class dominates (e.g., 99% vs. 1%), accuracy is misleading. A naive classifier predicting the majority class achieves 99% accuracy.

Strategies for imbalanced data:

1. **Resampling:** oversample the minority class (SMOTE creates synthetic examples) or undersample the majority.
2. **Cost-sensitive learning:** weight the loss function inversely to class frequency.
3. **Threshold tuning:** adjust the decision threshold from 0.5 to a value that balances precision and recall.
4. **Anomaly detection:** treat the minority class as anomalies and use one-class methods.

Principle 5.1 (Prevalence and Prior Probability)**.** The model's predicted probabilities should be calibrated to reflect the true class prevalence. Platt scaling and isotonic regression adjust probabilities post-hoc.

5.8 Exercises

Exercise 5.1. *Train a logistic regression classifier on a synthetic 2D dataset with two classes. Plot the decision boundary and the predicted probabilities as a contour plot. How does the boundary change with L2 regularization strength?*

Exercise 5.2. *You have a dataset with 95% negative class and 5% positive class. A classifier achieves 94% accuracy. Is this good? Compute precision, recall, and F_1 if the confusion matrix is: $TP = 300$, $FN = 200$, $FP = 100$, $TN = 9400$.*

Exercise 5.3. *Compare LDA and logistic regression on a dataset where the true class-conditional distributions are Gaussians with different covariance matrices. Which method recovers the Bayes-optimal boundary? Why?*

Exercise 5.4. *Explain why decision trees have high variance. Use the concept of hierarchical partitioning: how many possible trees exist for n points in $\mathbb{R}^2$?*

Chapter 6

Feature Engineering and Selection

"The single most important thing you can do to improve your model is to create better features."

Learning Goals

- Understand why feature engineering is often the highest-leverage activity in applied ML.
- Apply common feature transformations: scaling, encoding, binning, interaction terms.
- Handle text, categorical, temporal, and missing data effectively.
- Select features using filter, wrapper, and embedded methods.
- Interpret feature importance using SHAP and permutation methods.

6.1 Why Feature Engineering Matters

Features are the input representation for a model. The quality of this representation determines the ceiling on model performance. A linear model with excellent features can outperform a neural network with raw inputs.

Principle 6.1 (Data-Centric AI)**.** Improving data quality and feature engineering often yields larger gains than improving model architecture or hyperparameter tuning [2].

6.2 Feature Transformation

6.2.1 Scaling

Many models are sensitive to feature scales:

- **Standardization (Z-score):** $x' = \frac{x-\mu}{\sigma}$. Centres at 0, scales to unit variance.
- **Min–Max scaling:** $x' = \frac{x-\min(x)}{\max(x)-\min(x)}$. Maps to $[0, 1]$.
- **Robust scaling:** $x' = \frac{x-\text{median}(x)}{\text{IQR}(x)}$. Resistant to outliers.

Decision trees and tree ensembles are scale-invariant. Linear models, SVMs, and neural networks are not.

6.2.2 Log Transformation

For right-skewed distributions (income, population, counts):

$$x' = \log(x + 1)$$

This compresses the dynamic range and can make relationships more linear.

6.2.3 Binning (Discretization)

Converting a continuous feature into discrete bins can capture non-linear effects:

Listing 6.1: Equal-width binning

```
import pandas as pd
pd.cut(age, bins=[0, 18, 35, 60, 100],
       labels=['child', 'young', 'adult', 'senior'])
```

- **Equal-width:** bins of equal range.
- **Equal-frequency (quantile):** bins with equal number of samples.
- **Adaptive:** bin boundaries chosen by domain knowledge or tree splits.

6.2.4 Interaction Features

When the effect of one feature depends on another, include their product:

$$x_3 = x_1 \times x_2$$

Including all pairwise interactions expands d features to $d + d(d-1)/2$. Tree-based models discover interactions automatically; linear models require explicit construction.

6.3 Encoding Categorical Features

6.3.1 One-Hot Encoding

Create a binary column for each category:

$$\text{color} = \{\text{red}, \text{green}, \text{blue}\} \rightarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Dummy encoding drops one column ($K - 1$ columns for K categories) to avoid perfect multicollinearity.

6.3.2 Target Encoding

Replace each category with the mean target value for that category:

$$x'_{\text{cat}} = \frac{1}{n_{\text{cat}}} \sum_{i \in \text{cat}} y_i$$

This is powerful but prone to overfitting, especially for rare categories. Smooth with a prior:

$$x'_{\text{cat}} = \frac{n_{\text{cat}} \bar{y}_{\text{cat}} + \lambda \bar{y}_{\text{global}}}{n_{\text{cat}} + \lambda}$$

6.3.3 Frequency Encoding

Replace category with its count in the training set. Useful when frequency correlates with the target.

6.4 Text Features

6.4.1 TF–IDF

Term Frequency–Inverse Document Frequency transforms text into numerical vectors:

$$\text{TF–IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{\text{DF}(t)}$$

- $\text{TF}(t, d)$: frequency of term t in document d.
- $\text{DF}(t)$: number of documents containing term t.
- N: total documents.

TF–IDF down-weights common words ("the", "and") and up-weights rare, informative terms.

6.4.2 N-grams and Hashing

- **Unigrams:** individual words.
- **Bigrams:** pairs of consecutive words.
- **Feature hashing:** map arbitrary features to a fixed-size vector using a hash function (no dictionary needed).

6.5 Temporal and Date Features

Raw timestamps are rarely useful. Engineer:

- Hour of day, day of week, month, quarter.
- Is weekend? Is holiday?
- Time since last event, rolling statistics (7-day mean, 30-day sum).
- Cyclical encoding: $\sin(2\pi \cdot \text{hour}/24)$, $\cos(2\pi \cdot \text{hour}/24)$.

6.6 Handling Missing Data

Strategies, in order of preference:

1. **Remove:** drop rows or columns with excessive missingness ($> 50\%$).
2. **Mean/median imputation:** fill with the feature mean. Simple but naive.
3. **Model-based imputation:** predict missing values from observed ones (MICE, KNN imputer).
4. **Indicator:** add a binary column "is missing".
5. **Distributional:** sample from the observed distribution of the feature.

Principle 6.2 (Missing Not at Random)**.** If the fact that a value is missing is informative about the target (e.g., survey non-response correlates with income), then proper treatment of missingness is critical. An indicator feature captures this signal.

6.7 Feature Selection

Feature selection reduces dimensionality, improves generalization, and speeds up training.

6.7.1 Filter Methods

Rank features independently of the model:

- **Correlation:** $|\rho(X_j, y)|$ for regression; ANOVA F-statistic for classification.

- **Mutual information:** $I(X_j; Y) = \sum_{x_j} \sum_y p(x_j, y) \log \frac{p(x_j, y)}{p(x_j)p(y)}$
- **Variance threshold:** remove features with near-zero variance.

6.7.2 Wrapper Methods

Evaluate feature subsets by training a model on each:

- **Forward selection:** start empty, add features one by one.
- **Backward elimination:** start with all features, remove the least important.
- **Recursive feature elimination (RFE):** train, rank, remove bottom features, repeat.

Wrapper methods are computationally expensive but account for feature interactions.

6.7.3 Embedded Methods

Feature selection integrated into model training:

- **Lasso (L1 regularization):** drives irrelevant coefficients to zero [3].
- **Tree importance:** feature importance from tree splits.
- **Random forest importance:** mean decrease in impurity.

6.8 Feature Importance and Interpretability

6.8.1 Permutation Importance

Randomly shuffle a feature's values and measure the drop in performance:

$$\text{Importance}(X_j) = \text{Score}(\text{original}) - \text{Score}(\text{shuffled } X_j)$$

This measures how much the model relies on X_j, regardless of the model type.

6.8.2 SHAP Values

SHAP (SHapley Additive exPlanations) [4] decomposes a prediction into additive feature contributions based on cooperative game theory [5]:

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^{d} \phi_j$$

- ϕ_0: baseline prediction (mean over training data).
- ϕ_j: contribution of feature j for this specific prediction.
- SHAP values satisfy desirable properties: local accuracy, missingness, consistency.

SHAP provides both global feature importance (mean $|\phi_j|$ across all data) and local explanations (why a specific prediction was made).

6.9 The Feature Engineering Pipeline

Listing 6.2: Feature engineering pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestRegressor

numeric_features = ['age', 'income', 'hours']
categorical_features = ['education', 'occupation']
```

```
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(), categorical_features)
])

pipeline = Pipeline([
    ('preprocess', preprocessor),
    ('model', RandomForestRegressor())
])

pipeline.fit(X_train, y_train)
```

6.10 Exercises

Exercise 6.1. *Take a continuous feature with a right-skewed distribution (e.g., income). Compare a model with the raw feature vs.* $\log(\textit{income})$*. How do the residuals change? Which version produces more interpretable coefficients?*

Exercise 6.2. *Apply one-hot encoding and target encoding to the same categorical feature with 50 categories. Train a linear model and a tree-based model. Compare the results. Why does target encoding generally help linear models more than tree models?*

Exercise 6.3. *Given 100 features and 500 samples, use Lasso regression to select features. Vary the regularization parameter* λ *and plot the number of non-zero coefficients vs.* λ*. How does cross-validation choose* λ*?*

Exercise 6.4. *Using SHAP, explain why a specific test point received its prediction. List the top 3 features and their contributions. Are these globally important features or locally specific?*

Chapter 7

Dimensionality Reduction

"The curse of dimensionality is not about computation. It is about geometry. In high dimensions, all points are far apart."

Learning Goals

- Explain the curse of dimensionality and why high-dimensional data is problematic.
- Derive principal component analysis from the variance-maximisation perspective.
- Understand the SVD formulation of PCA and its connection to feature engineering.
- Compare linear (PCA) and non-linear (t-SNE, UMAP) dimensionality reduction.
- Apply PCA for noise reduction, visualization, and pre-processing.

7.1 The Curse of Dimensionality

As dimensionality d increases, the volume of space grows exponentially. Data becomes sparse, distances converge, and models require exponentially more samples.

Example 7.1 (Concentration of Distances)**.** For n points uniformly distributed in $[0,1]^d$, the ratio of the nearest-neighbor distance to the farthest-neighbor distance approaches 1 as d grows:

$$\frac{\text{dist}_{\text{nearest}}}{\text{dist}_{\text{farthest}}} \to 1$$

In high dimensions, the concept of "nearest neighbor" loses meaning.

Consequences:

- KNN classifiers perform poorly above $d \approx 20$ without feature selection.
- Density estimation requires $n \propto 2^d$ samples.
- Overfitting risk increases as d approaches n.

7.2 Principal Component Analysis

PCA is the most widely used linear dimensionality reduction method. It finds a sequence of orthogonal directions that maximize the variance of the projected data.

Definition 7.1 (PCA Objective)**.** Find $\mathbf{w}_1$ (first principal component) that maximizes the variance of $\mathbf{X}\mathbf{w}_1$:

$$\mathbf{w}_1 = \arg\max_{\|\mathbf{w}\|=1} \text{Var}(\mathbf{X}\mathbf{w}) = \arg\max_{\|\mathbf{w}\|=1} \mathbf{w}^\top \mathbf{\Sigma} \mathbf{w}$$

where $\mathbf{\Sigma} = \frac{1}{n}\mathbf{X}^\top\mathbf{X}$ is the covariance matrix (assuming centred data).

7.2.1 Eigenvalue Solution

The solution is the eigenvector of $\boldsymbol{\Sigma}$ with the largest eigenvalue:

$$\boldsymbol{\Sigma}\mathbf{w}_1 = \lambda_1 \mathbf{w}_1$$

Subsequent components $\mathbf{w}_2, \ldots, \mathbf{w}_k$ are the next eigenvectors, orthogonal to all previous ones.

7.2.2 PCA via SVD

PCA is more numerically stable when computed via the SVD of the data matrix:

$$\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$$

- Columns of $\mathbf{V}$: principal component directions (same as eigenvectors of $\mathbf{X}^\top\mathbf{X}$).
- Singular values σ_j: related to eigenvalues by $\lambda_j = \sigma_j^2/n$.
- Principal component scores: $\mathbf{Z} = \mathbf{X}\mathbf{V}$.
- Variance explained by component j: $\sigma_j^2 / \sum_{i=1}^{d} \sigma_i^2$.

7.2.3 Choosing the Number of Components

A scree plot shows the variance per component. Common criteria:

- **Kaiser rule:** keep components with eigenvalue > 1 (for correlation matrix).
- **Cumulative variance:** keep enough components to explain 90% or 95% of variance.
- **Elbow method:** look for the knee in the scree plot.

7.2.4 PCA as Feature Engineering

PCA creates new features (principal components) that are:

- Uncorrelated (orthogonal).
- Ordered by variance (information content).
- Linear combinations of original features.
- Useful for visualization (first 2–3 components).
- Effective denoising (discard low-variance components).

Principle 7.1 (PCA for Linear Models)**.** Since PCA components are uncorrelated, linear models fit on PCA-transformed data have no multicollinearity. This stabilizes coefficient estimates and can improve generalization when $d \gg n$.

7.3 Factor Analysis

Factor analysis is similar to PCA but with a different goal: model the covariance structure.

$$\mathbf{x} = \boldsymbol{\mu} + \mathbf{L}\mathbf{f} + \boldsymbol{\epsilon}$$

- $\mathbf{f} \sim \mathcal{N}(0, \mathbf{I})$: latent factors ($k \ll d$).
- $\mathbf{L} \in \mathbb{R}^{d \times k}$: factor loadings (how each feature relates to each factor).
- $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \boldsymbol{\Psi})$: unique variances (diagonal).

Unlike PCA, factor analysis explicitly models measurement error and assumes the observed correlations arise from a small number of common causes.

7.4 t-Distributed Stochastic Neighbor Embedding

t-SNE is a non-linear method primarily used for visualization in 2D or 3D.

- **Local structure preservation:** points close in high-dimensional space remain close in low-dimensional space.
- **Probability-based:** convert distances to conditional probabilities $p_{j|i}$, then minimize KL divergence between high-dim and low-dim distributions.
- **Heavy-tailed distribution:** uses a Student-t distribution in the low-dimensional space to alleviate the crowding problem.

Listing 7.1: t-SNE for visualization

```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, perplexity=30, random_state=0)
X_2d = tsne.fit_transform(X)
```

Key hyperparameter: `perplexity` (roughly the effective number of neighbors). Typical range: 5–50.

7.4.1 Limitations of t-SNE

- Non-deterministic: different runs give different results.
- Global structure (distances between clusters) is not preserved.
- Computationally expensive: $O(n^2)$.
- Cannot embed new points (transductive, not inductive).

7.5 UMAP

Uniform Manifold Approximation and Projection (UMAP) addresses some of t-SNE's limitations:

- Faster: $O(n \log n)$ via approximate nearest neighbors.
- Better global structure preservation.
- Supports embedding new points after training.
- Based on manifold theory and fuzzy simplicial sets.

UMAP is preferred over t-SNE for large datasets and when reproducibility matters.

7.6 Autoencoders

Neural network-based dimensionality reduction. An autoencoder learns to reconstruct its input through a bottleneck layer of dimension $k \ll d$:

$$\mathbf{h} = f(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e)$$

$$\hat{\mathbf{x}} = g(\mathbf{W}_d \mathbf{h} + \mathbf{b}_d)$$

The bottleneck $\mathbf{h}$ is a non-linear low-dimensional representation. Autoencoders can capture manifolds that linear methods (PCA) cannot.

7.7 Choosing a Method

Method	Type	Best for	Complexity

PCA	Linear	Pre-processing, denoising	$O(\min(nd^2, dn^2))$
Factor analysis	Linear (probabilistic)	Covariance modelling	$O(ndk)$
t-SNE	Non-linear	Visualization (2D/3D)	$O(n^2)$
UMAP	Non-linear	Visualization, large data	$O(n \log n)$
Autoencoder	Non-linear (neural)	Feature learning	$O(ndh)$

7.8 Exercises

Exercise 7.1. *Generate 1000 points uniformly in* $[0, 1]^d$ *for* $d = 2, 10, 50, 100$. *For each dimension, compute the ratio of maximum to minimum pairwise distance. At what* d *does the ratio approach 1?*

Exercise 7.2. *Apply PCA to the Iris dataset. Compute the variance explained by each component. How many components capture* 95% *of the variance? Visualize the data projected onto the first two principal components, colored by species.*

Exercise 7.3. *Compare PCA and t-SNE on the MNIST dataset. Plot the 2D embeddings side by side. Which method produces better separation of digit classes? Which method preserves the global structure (e.g., the fact that 3 and 8 are more similar than 3 and 0)?*

Exercise 7.4. *Use PCA as a pre-processing step for linear regression on a dataset with* $d > n$. *Compare test error with and without PCA. How many components minimize test error? Why does PCA help here?*

Part III

Practical Machine Learning

Chapter 8

Model Evaluation and Validation

"A model is not good because it fits the training data well. A model is good because it predicts unseen data well."

Learning Goals

- Design proper train–validation–test splits for reliable evaluation.
- Apply cross-validation and understand its bias–variance tradeoff.
- Diagnose overfitting using learning curves.
- Avoid common evaluation pitfalls: data leakage, selection bias, and multiple testing.

8.1 The Evaluation Problem

The central question in model evaluation: *how will this model perform on new, unseen data?* We cannot answer this by testing on the data used to train the model, because the model has memorized it.

Definition 8.1 (Generalization Error)**.** The **generalization error** is the expected loss on new data drawn from the same distribution as the training data:

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x},y)\sim P}[L(y, f(\mathbf{x}))]$$

We estimate this by holding out a test set.

8.2 Train–Validation–Test Split

Always split data into three disjoint sets:

- **Training set** (typically 60–80%): used to fit model parameters.
- **Validation set** (10–20%): used for hyperparameter tuning and model selection.
- **Test set** (10–20%): used *exactly once* to report final performance.

Principle 8.1 (The Golden Rule of Testing)**.** The test set may influence model selection *zero times*. Once you evaluate on the test set, that number is final. Any iterative improvement based on test scores invalidates the estimate.

8.3 Cross-Validation

When data is limited, a single validation split is unreliable. Cross-validation averages over multiple splits.

8.3.1 K-Fold Cross-Validation

1. Split data into K equal folds (typically $K = 5$ or 10).
2. For fold k: train on all folds except k, validate on fold k.
3. Report the average performance across folds.

Listing 8.1: K-fold cross-validation

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5,
                         scoring='neg_mean_squared_error')
print(f'MSE: {-scores.mean():.3f} +/- {scores.std():.3f}')
```

8.3.2 Bias–Variance of Cross-Validation

- $K = 2$: low computational cost, high bias (training on only half the data).
- $K = n$ (leave-one-out, LOOCV): low bias, high variance, high computation.
- $K = 10$: standard choice; balances bias and variance.

8.3.3 Stratified Cross-Validation

For classification, preserve class proportions in each fold:

$$\frac{n_k^{(\text{fold})}}{n^{(\text{fold})}} \approx \frac{n_k}{n}$$

8.4 Learning Curves

Plot training error and validation error as a function of training set size:

- **High bias:** both curves converge to a high error; adding more data does not help.
- **High variance:** training error is low, validation error is high, and the gap persists as data increases.

Principle 8.2 (Diagnosing with Learning Curves)**.** If the validation curve plateaus well above the training curve, you have high bias (the model is too simple). If the curves are far apart and the training curve is near zero, you have high variance (the model is too complex).

8.5 Bootstrap and Confidence Intervals

The bootstrap provides confidence intervals for performance metrics without parametric assumptions:

1. Resample the dataset with replacement B times (e.g., $B = 1000$).
2. For each bootstrap sample, train and evaluate the model.
3. The 2.5th and 97.5th percentiles of the B scores form a 95% confidence interval.

8.6 Hyperparameter Tuning

Hyperparameters control model complexity and are set before training:

Model	Hyperparameters	Effect
Ridge regression	α	Regularization strength
Random forest	`n_estimators`, `max_depth`	Ensemble size, tree depth
SVM	C, γ	Margin, RBF width
Neural network	`lr`, `batch_size`, `layers`	Learning rate, architecture
XGBoost	`eta`, `max_depth`, `subsample`	Learning rate, tree params

8.6.1 Grid Search

Exhaustively evaluate all combinations of specified hyperparameter values. With K-fold CV, this requires $K \times \prod_j |\Theta_j|$ model fits.

8.6.2 Random Search

Sample hyperparameter values from distributions. More efficient than grid search when only a few hyperparameters matter [6].

Listing 8.2: Random search with CV

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import uniform, randint

param_dist = {
    'n_estimators': randint(50, 500),
    'max_depth': randint(3, 20),
    'learning_rate': uniform(0.01, 0.3)
}
search = RandomizedSearchCV(model, param_dist,
                            n_iter=100, cv=5)
search.fit(X_train, y_train)
```

8.7 Common Pitfalls

8.7.1 Data Leakage

Information from the test set leaks into training:

- Fitting scaler or imputer *before* splitting.
- Using future data to predict the past (time series).
- Feature selection on the full dataset before splitting.

Principle 8.3 (No Peeking)**.** Any transformation that uses information from outside the training set is data leakage. Fit all transformations (scaling, imputation, PCA, feature selection) on the training set *only*, then apply to validation/test sets.

8.7.2 Multiple Testing and p-Hacking

Evaluating many models on the same test set inflates the chance of finding a spuriously good result. If you test 20 models, you expect one to achieve $p < 0.05$ by chance.

8.7.3 Selection Bias

If the training data is not representative of the deployment distribution, the evaluation is optimistic. Common causes: convenience sampling, temporal drift, non-response bias.

8.8 Comparing Models Statistically

To compare two models A and B:

- **Paired t-test:** compute the per-fold difference $d_k = \text{error}_k^A - \text{error}_k^B$ and test $\bar{d} \neq 0$.
- **McNemar's test:** for classification, compare the number of points where A is correct and B is wrong, vs. vice versa.
- **Bayesian comparison:** use a Bayesian hierarchical model to estimate the probability that A is better than B.

8.9 Exercises

Exercise 8.1. *Take a dataset and perform a 70/15/15 train/validation/test split. Train a model, tune hyperparameters on the validation set, and evaluate on the test set once. Now suppose you repeat the tuning step 10 times, each time evaluating on the same test set. What happens to your estimate of generalization error?*

Exercise 8.2. *Generate a learning curve for a decision tree on a synthetic dataset. Vary the training set size from 10 to 500. Plot training MSE and validation MSE. At what point does the validation error stop decreasing? What does this tell you about the value of additional data?*

Exercise 8.3. *Design a cross-validation strategy for a time series forecasting task. Why is standard K-fold CV inappropriate? What alternative would you use?*

Exercise 8.4. *You accidentally scaled the full dataset before splitting into train and test. How does this affect the test error estimate? Quantify the bias: is it optimistic or pessimistic?*

Chapter 9

Regularization and Ensemble Methods

"The best single model is rarely as good as a collection of mediocre models."

Learning Goals

- Understand regularization as a mechanism to control model complexity and prevent overfitting.
- Derive ridge regression and lasso, and contrast L1 vs. L2 regularization.
- Explain how bagging reduces variance without increasing bias.
- Understand boosting as adaptive reweighting of misclassified points.
- Build random forests and gradient-boosted trees for structured data.

9.1 Regularization

Regularization adds a penalty to the loss function to discourage overly complex models:

$$L_{\text{reg}}(\mathbf{w}) = L(\mathbf{w}) + \lambda \cdot P(\mathbf{w})$$

9.1.1 Ridge Regression (L2)

$$P(\mathbf{w}) = \|\mathbf{w}\|_2^2 = \sum_{j=1}^{d} w_j^2$$

$$\hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

Ridge shrinks coefficients toward zero but never to exactly zero. It works well when many features have small effects.

Definition 9.1 (Bias–Variance in Ridge)**.** Ridge increases bias (coefficients are biased toward zero) but decreases variance (coefficients are more stable). The total prediction error can decrease if the variance reduction outweighs the bias increase [7].

9.1.2 Lasso (L1)

$$P(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_{j=1}^{d} |w_j|$$

Lasso drives some coefficients to exactly zero, performing automatic feature selection [3]. This makes it useful when only a few features are relevant.

9.1.3 Elastic Net

Combines L1 and L2 penalties:

$$P(\mathbf{w}) = \alpha\|\mathbf{w}\|_1 + (1-\alpha)\|\mathbf{w}\|_2^2$$

Elastic net handles groups of correlated features better than lasso (which tends to pick one from a group).

9.1.4 The Regularization Path

As λ increases from 0 to ∞:

- Ridge: coefficients smoothly approach 0.
- Lasso: coefficients shrink linearly; some hit 0 at finite λ.

Cross-validation selects λ to minimize validation error.

9.2 Bagging

Bootstrap Aggregating (bagging) reduces variance by averaging many models trained on bootstrap samples:

1. Draw B bootstrap samples from the training data.
2. Train a model on each sample.
3. Average predictions (regression) or majority vote (classification).

Theorem 9.1 (Variance Reduction)**.** If B models each have variance σ^2 and pairwise correlation ρ, the ensemble variance is:

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

As $B \to \infty$, variance converges to $\rho\sigma^2$. Bagging is most effective when models are uncorrelated ($\rho \approx 0$).

9.3 Random Forests

Random forests improve on bagged trees by decorrelating the trees [8]:

- Each tree is trained on a bootstrap sample.
- At each split, only a random subset of m features is considered ($m \approx \sqrt{d}$ for classification, $d/3$ for regression).
- This decorrelates the trees, reducing ρ and thus variance.

Listing 9.1: Random forest

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=500,
    max_features='sqrt',
```

```
    min_samples_leaf=5,
    n_jobs=-1
)
rf.fit(X_train, y_train)
```

Random forests are among the best off-the-shelf methods for structured (tabular) data. They handle non-linearity, interactions, missing values, and mixed data types.

9.4 Boosting

Boosting trains models sequentially, each focusing on the mistakes of the previous ones.

9.4.1 AdaBoost

Adaptive Boosting assigns weights to training points:

1. Initialize weights $w_i = 1/n$.
2. For $t = 1, \ldots, T$:
   - Train a weak learner f_t on weighted data.
   - Compute weighted error ϵ_t.
   - Set $\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$.
   - Increase weights for misclassified points: $w_i \leftarrow w_i e^{\alpha_t \mathbb{1}[y_i \neq f_t(\mathbf{x}_i)]}$.
3. Ensemble: $\hat{y} = \text{sign}(\sum_t \alpha_t f_t(\mathbf{x}))$.

9.4.2 Gradient Boosting

Generalizes boosting to any differentiable loss function:

$$f_{t+1}(\mathbf{x}) = f_t(\mathbf{x}) + \eta \cdot h_t(\mathbf{x})$$

where h_t is a weak learner (typically a shallow tree) fit to the *negative gradient* of the loss:

$$h_t \approx -\nabla_f L(y, f_t(\mathbf{x}))$$

For squared error loss, the gradient is just the residual $y - f_t(\mathbf{x})$, so gradient boosting is "fitting to the errors".

9.4.3 XGBoost, LightGBM, CatBoost

Modern gradient boosting implementations add:

- **XGBoost [9]:** regularization, second-order gradients, weighted quantile sketch.
- **LightGBM:** Gradient-based One-Side Sampling (GOSS), Exclusive Feature Bundling (EFB), leaf-wise tree growth.
- **CatBoost:** ordered boosting (handles categorical features), symmetric trees.

Listing 9.2: XGBoost

```
import xgboost as xgb

model = xgb.XGBRegressor(
    n_estimators=1000,
    learning_rate=0.01,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8
)
model.fit(X_train, y_train,
```

```
        eval_set=[(X_val, y_val)],
        early_stopping_rounds=50)
```

9.5 Stacking

Stacked generalization combines multiple different models via a meta-learner:

1. Split training data into K folds.
2. For each base model m, generate out-of-fold predictions for all training points.
3. Train a meta-model on these predictions as features.

Stacking can improve over any single model but risks overfitting if not done carefully.

9.6 Practical Guidelines

Scenario	**Recommended method**	**Why**
$n \gg d$, linear relationship	Ridge or Lasso	Simple, interpretable
$d \gg n$, sparse signal	Lasso	Feature selection
Tabular data, $n \sim 10^3$–10^5	XGBoost / LightGBM	Best predictive accuracy
Heterogeneous data types	Random forest	Robust, no tuning needed
Need calibrated probabilities	Gradient boosting + Platt	Better calibration than RF
Very large n ($> 10^6$)	Linear model or LightGBM	Computational constraints

Principle 9.1 (Swets' Law)**.** The performance of a well-tuned tree ensemble and a well-tuned neural network on tabular data is often similar. The main difference is in computational cost, interpretability, and data efficiency.

9.7 Exercises

Exercise 9.1. *Fit ridge regression on a dataset with correlated features. Plot the coefficient paths as λ varies. Compare with lasso: which coefficients go to zero first? Why?*

Exercise 9.2. *Train a single decision tree, a random forest with 100 trees, and an XGBoost model on the same dataset. Compare training error and test error. Which ensemble reduces variance most? Which reduces bias most?*

Exercise 9.3. *Implement AdaBoost from scratch (or using scikit-learn's `AdaBoostClassifier`) with decision stumps (depth-1 trees) as weak learners. Plot the training and test error as a function of the number of boosting rounds.*

Exercise 9.4. *Explain why random forests use both bootstrap sampling and random feature subsets. What would happen if you used only one of these sources of randomness?*

Chapter 10

The Machine Learning Pipeline

"In production, the model is 5% of the code. The pipeline is the other 95%."

Learning Goals

- Design a complete ML pipeline from raw data to deployed model.
- Integrate feature engineering, model selection, and evaluation into a reproducible workflow.
- Understand MLOps concerns: versioning, monitoring, drift detection.
- Recognize when ML is the right tool and when simpler alternatives suffice.

10.1 The End-to-End Pipeline

A production ML system involves far more than training a model:

Listing 10.1: The ML pipeline stages

```
1. Data ingestion -- collect, validate, store raw data
2. Data cleaning -- handle missing values, outliers, duplicates
3. Feature engineering-- transform, encode, select features
4. Model training -- select algorithm, tune hyperparameters
5. Model evaluation -- validate, compare, select final model
6. Model deployment -- serve predictions via API or batch
7. Monitoring -- track performance, detect drift
8. Retraining -- update model with new data
```

10.2 Data Ingestion and Validation

Before any modelling, ensure data quality:

- **Schema validation:** check column names, types, value ranges.
- **Statistical validation:** check mean, variance, null ratios against expected ranges.
- **Integrity checks:** unique key constraints, referential integrity.
- **Temporal validation:** ensure no leakage from future to past.

Tools: Great Expectations, Deequ, Pandera.

10.3 Data Cleaning

10.3.1 Outlier Detection

Statistical methods for identifying anomalous points:

$$\text{Is outlier if } |x - \mu| > 3\sigma \quad \text{(Z-score method)}$$

$$\text{Is outlier if } x < Q_1 - 1.5 \cdot \text{IQR} \text{ or } x > Q_3 + 1.5 \cdot \text{IQR}$$

For high-dimensional data, use isolation forest, LOF, or autoencoder reconstruction error.

10.3.2 Duplicate Detection

Duplicate rows inflate the effective sample size and bias the model. Identify exact duplicates and near-duplicates (using Jaccard similarity or MinHash).

10.4 Feature Store

A feature store is a centralized repository for feature definitions and computations:

- **Online store:** low-latency feature serving for real-time predictions (Redis, DynamoDB).
- **Offline store:** batch feature computation for training (Parquet files, BigQuery).
- **Feature registry:** metadata about feature definitions, owners, and dependencies.
- **Point-in-time correctness:** ensure training features use only information available at prediction time.

Principle 10.1 (Point-in-Time Correctness)**.** When joining features for training, use only data that was available at the time of prediction. A common bug: using future information (e.g., a feature computed from the full dataset) that is not available in production.

10.5 Experiment Tracking

Every ML experiment should be recorded:

- **Data:** dataset version, hash, split indices.
- **Code:** git commit, hyperparameters, model architecture.
- **Results:** metrics (training, validation, test), learning curves.
- **Artifacts:** model weights, scalers, encoders, preprocessing config.

Tools: MLflow, Weights & Biases, Neptune, DVC.

10.6 Model Deployment

10.6.1 Batch Inference

Predict on a schedule (hourly, daily). Simple: read data, apply model, write predictions to a database.

Scalability:partition data by date, process in parallel with Spark or Dask.

10.6.2 Real-Time Inference

Serve predictions via an API with millisecond latency:

Listing 10.2: Real-time model serving outline

```
from flask import Flask, request, jsonify

app = Flask(__name__)
model = load_model('model.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    features = request.json['features']
    probs = model.predict_proba([features])[0]
    return jsonify({
        'prediction': int(probs[1] > 0.5),
        'probability': float(probs[1])
    })
```

10.6.3 Streaming Inference

Predict on individual events as they arrive (Kafka, Kinesis):

$$\text{Event} \rightarrow \text{Feature computation} \rightarrow \text{Model prediction} \rightarrow \text{Action}$$

10.7 Monitoring and Drift Detection

10.7.1 Concept Drift

The relationship $P(y \mid \mathbf{x})$ changes over time:

- **Sudden drift:** abrupt change (e.g., new regulation).
- **Gradual drift:** slow change (e.g., aging population).
- **Seasonal drift:** recurring pattern (e.g., holiday shopping).

10.7.2 Data Drift

The input distribution $P(\mathbf{x})$ changes:

$$D_{\text{KL}}(P_{\text{train}}(\mathbf{x}) \parallel P_{\text{current}}(\mathbf{x})) > \text{threshold}$$

Monitor per-feature statistics (mean, variance, quantiles) over sliding windows.

10.7.3 Performance Monitoring

Track the same metrics used during evaluation:

- Accuracy, precision, recall (requires labels, which may be delayed).
- Proxy metrics when labels are unavailable (prediction distribution, feature statistics).
- Business metrics: revenue, conversion rate, user engagement.

10.8 Retraining Strategies

Strategy	Trigger	Trade-off

Scheduled	Time-based (weekly, monthly)	Simple but wasteful
Performance-triggered	Metric drops below threshold	Requires labeling pipeline
Drift-triggered	Feature/target distribution shift	No labels needed, early warning
Online learning	Continuous update per batch	Fast adaptation, complex ops

10.9 When Not to Use ML

Not every problem needs machine learning:

- **Simple heuristics:** if $y = \text{sign}(x_1 > 5)$, write a rule.
- **Lookup tables:** if the mapping is fixed and enumerable.
- **Linear regression:** if the relationship is linear and $d < 10$.
- **When data is scarce:** ML needs data. With $n < 100$, use simple methods or domain knowledge.

Principle 10.2 (Occam's Razor in ML)**.** Among models with similar performance, choose the simplest. Simpler models are easier to debug, deploy, explain, and maintain. Start with linear regression before trying XGBoost; start with a baseline heuristic before training a neural network.

10.10 Ethical Considerations

- **Fairness:** Does the model perform equally across demographic groups? Check disparate impact: $\frac{P(\hat{y}=1|A)}{P(\hat{y}=1|B)}$.
- **Bias in data:** If the training data reflects historical discrimination, the model will perpetuate it.
- **Explainability:** Stakeholders need to understand why a prediction was made. Use SHAP, LIME, or inherently interpretable models (GLMs, decision trees).
- **Privacy:** Models can memorize training data. Use differential privacy or federated learning for sensitive data.

10.11 The ML Pipeline in Practice

Listing 10.3: Complete pipeline sketch

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

pipeline = Pipeline([
    ('preprocess', ColumnTransformer([
        ('num', StandardScaler(), numeric_cols),
        ('cat', OneHotEncoder(), categorical_cols)
    ])),
    ('model', RandomForestRegressor())
])

# Train with cross-validation
scores = cross_val_score(
    pipeline, X_train, y_train,
```

```
    cv=5, scoring='neg_root_mean_squared_error'
)
print(f'CV RMSE: {-scores.mean():.3f} +/- {scores.std():.3f}')

# Final fit
pipeline.fit(X_train, y_train)

# Evaluate once
y_pred = pipeline.predict(X_test)
test_rmse = mean_squared_error(y_test, y_pred, squared=False)
print(f'Test RMSE: {test_rmse:.3f}')
```

10.12 Exercises

Exercise 10.1. *Design an ML pipeline for a churn prediction system. List each stage, the tools you would use, and how you would handle data leakage between training and inference.*

Exercise 10.2. *Monitor a deployed model for drift. Compute the population stability index (PSI) for each feature:*

$$PSI = \sum_k (p_k^{train} - p_k^{current}) \log \frac{p_k^{train}}{p_k^{current}}$$

where p_k is the proportion of values in bin k. A $PSI > 0.1$ indicates drift.

Exercise 10.3. *Your production model's accuracy drops from 92% to 85% over a week. Outline a debugging plan. How would you distinguish between data drift, concept drift, and a software bug?*

Exercise 10.4. *A stakeholder asks you to build an ML model to predict employee performance from their resume. Identify three ethical concerns with this task. What data would you need to audit for fairness?*

Bibliography

[1] George E. P. Box. Science and statistics. *Journal of the American Statistical Association*, 71(356):791–799, 1976.

[2] Andrew Ng. *Machine Learning Yearning*. Self-published, 2024.

[3] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B*, 58(1):267–288, 1996.

[4] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. pages 4765–4774, 2017.

[5] Lloyd S. Shapley. A value for n-person games. *Contributions to the Theory of Games*, 2 (28):307–317, 1953.

[6] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(1):281–305, 2012.

[7] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[8] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[9] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *ACM SIGKDD*, pages 785–794, 2016.